Learning Experience F.2

Ethan Davidson and Patrick Rooney

ECE 2544

Fall 2024

Section I: Specifications

1. Objective:
    - The main objective of this learning experience is to implement a function unit and trace the execution of a small program on the Simple Computer. This will be done by creating a function unit, which is the same as the function unit from Learning Experience F.1, and creating a new Control Unit to control the Datapath. The Control Unit will be constructed by using instruction memory to fetch and execute a series of instructions. The overall objective of this learning experience is to implement a function unit in the Datapath and assign Opcodes to instructions that use the function unit.
2. Requirements and Constraints
    - We are only permitted to edit function_unit.v, instr_decoder.v, data.txt, instruction.txt and waveform files.
    - We are not permitted to modify port declarations.
    - Only structural and permitted dataflow Verilog constructs are permitted to be used.
    - Schematics or behavioral Verilog constructs are not permitted to be used.
    - All instructions must be completed in one clock cycle.
    - The data.txt file we submit must include the last four digits of our student ID numbers and data values outlined in the Completion section of the specification.
    - The instruction.txt file must include the sequence of instructions outlined in the Completion section of the specification.
    - Each instruction may only take one of the two formats outlined in Section 2.3 of the specification.
    - The original instructions outlined in Table 2.4 of the specification must still be functional and our new instructions and Opcodes must be in addition to these.

Section II: Design Approach

C.3 Design Approach

For our final circuit, we made modifications to the function unit from Learning Experience C.2 and we made modifications to the bus system from Learning Experience D.1 and combined them to make a function unit and bus system. To make both designs compatible with each other, we had to set the registers to be the inputs of the function unit. This allows the function unit to properly load values from the bus system. We also needed to make modifications to the system so data could be stored in the register system. We did this by using a 4x1 mux to select which register would be assigned to Operand A and Operand B for each operation. We then used a decoder to control when each register would load values, giving us the ability to control which values could be placed in specific registers. To implement our design, we used structural and permitted dataflow in our Verilog model. Another design approach we considered was modifying our bus system from Learning Experience D.2, rather than modifying our bus system from Learning Experience D.1. We ultimately decided against this, since Learning Experience D.2 uses tri state gates as the bus system design, and Learning Experience D.1 uses multiplexers as the bus system design. We decided to modify the bus system from Learning Experience D.1 because we have more experience working with multiplexers and felt more comfortable making a design that could meet the specifications of this project this way.

F.1 Design Changes

Overall, we did not make any major design changes to our function unit from Learning Experience C.3. One design change we made was increasing the number of wires from eight to sixteen for our wire vectors. We also changed how our shift functions work for rem8 and mult4 functions by changing which bits were moved to allow all sixteen bits to be affected by the shift. We also added eight full adders and sixteen muxes, so our adder circuit would be able to handle 16-bit values. The reason we made these design changes was to allow our function unit to handle two 16-bit operands and output a 16-bit result based on a given operation performed by the function unit.

F.2 Design Approach And Opcodes

Operation	15	14	13	12	11	10	9	Function Select				MB	MD	RW	MW	PL	JB	BC
Add	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
Subtract	0	0	0	0	0	0	1	0	0	0	1	0	0	1	0	0	0	1
Negative A	1	0	0	0	0	1	0	0	0	1	0	1	0	1	0	0	0	0
Mult 4	0	0	0	1	0	1	1	1	0	1	1	0	0	1	0	0	0	1
Mov A	0	0	0	0	1	0	1	0	1	0	1	0	0	1	0	0	0	1
Rem 8	0	0	0	1	1	0	0	1	1	0	0	0	0	1	0	0	0	0
Nor	0	0	0	0	1	1	1	0	1	1	1	0	0	1	0	0	0	1
Negative B	0	0	0	0	0	1	1	0	0	1	1	0	0	1	0	0	0	1
Dec A	1	0	0	0	1	0	0	0	1	0	0	1	0	1	0	0	0	0
Not A	1	0	0	0	1	1	0	0	1	1	0	1	0	1	0	0	0	0
And	0	0	0	1	0	0	0	1	0	0	0	0	0	1	0	0	0	0
Nand	0	0	0	1	0	0	1	1	0	0	1	0	0	1	0	0	0	1
Not B	0	0	0	1	0	1	0	1	0	1	0	0	0	1	0	0	0	0
ADD I	1	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0
SUB I	1	0	0	0	0	0	1	0	0	0	1	1	0	1	0	0	0	0
NOR I	1	0	0	0	1	1	1	0	1	1	1	1	0	1	0	0	0	1
Negative B	0	0	0	0	0	1	1	0	0	1	1	0	0	1	0	0	0	1
Decrement A	1	0	0	0	1	0	0	0	1	0	0	1	0	1	0	0	0	1
Not B	0	0	0	0	0	1	1	0	0	1	1	0	0	1	0	0	0	1

Figure 1.1 Opcode Table

For our final design, we were able to keep our same function unit and bus system from Learning Experience F.1, and we were able to properly implement this in the Datapath. Our design changes were centered around changing the control of the function unit from Word Control to Instruction Memory. In Learning Experience F.2, we were provided with a decoder in our Datapath. The decoder receives the Instruction Memory, decodes it, and “tells” the Simple Computer how to perform each operation. For Learning Experience F.2, our main design change was creating the contents of Instruction Memory to be performed by the decoder. We created Instruction memory by designing Opcodes. Since our instructions need to use Opcodes in order to be properly read by the decoder, our function select codes must be ‘built in’ to the Opcodes to achieve proper functionality. In order for our function select code to be compatible with the Opcode, function select bits [3:0] must be equal to Opcode bits [12:9]. This differs from what we learned in class about how decoders read Opcodes, as traditionally, function select bits [3:1] must be equal to Opcode bits [12:10] and function select bit [0] must be equal to ((Opcode[15]&Opcode[14])’&Opcode[9]). We were able to make the function select bits [3:0] equal to Opcode bits [12:9] by modifying the provided decoder. We modified the decoder by changing the function select 0 to be assigned to the operation 9 bit. This was the only way we could use our current simple computer in the designed decoder so that we could include every operation. We used the operation 9 bit because branch control is a don't care value throughout

all of our operations and wouldn't have a negative effect on the rest of the circuit if we used it. We can justify this design decision, as it was allowed in the specification, and we were not able to make function select bit [0] compatible with the decoder, as we ran into issues with ensuring mux B was also compatible with the decoder at the same time. This is because, as stated previously, function select bit [0] is equal to ((Opcode[15]&Opcode[14])'&Opcode[9]) and mux B select bit must be equal to Opcode [15], and these values can directly affect each other. In addition to ensuring proper functionality of our design, modifying the decoder also meant that we did not need to change our function unit circuit, allowing us to have a more efficient design, further justifying this design decision. Other influences on our Opcode design had to do with encoding the control word bits into our Opcode as well, to ensure our Opcode works properly with the decoder and the Datapath as a whole. In order for the control word bits to work properly with the Opcode MB must be Opcode[15], MD must be Opcode[13], JB must be Opcode[13], BC must be Opcode[9], RW must be Opcode[14]', MW must be (Opcode[14]&Opcode[15]') and PL must be (Opcode[14]&Opcode[15]. These are all of the factors that influenced the design of our Opcodes. Our Opcodes are shown in *Figure 1.1.*

Criteria Of Evaluation

Our criteria of evaluation were somewhat consistent with the criteria of evaluation outlined in the specification. One criterion we used to evaluate the success of our implementation was minimizing transistor count and propagation delays. This criterion was consistent with what was outlined in the specification, and was our main criteria of evaluation. Another criterion we used to evaluate our design was ensuring that our simulation matched the expected results shown in Table 2.6 of the Learning Experience F.2 specification sheet. This was also consistent with what was outlined in the specification and was a major criterion of evaluation. One of our main design decisions was keeping our function unit and bus system design from Learning Experience F.1. We decided to do this because the design met our criteria for evaluation and had efficient transistor counts and propagation delay. We also decided to change the decoder. We modified the decoder so that the function select bit [0] would be equal to Opcode bit [9]. The provided decoder was originally designed so the function select bit [0] would be equal to ((Opcode[15]&Opcode[14])'&Opcode[9]). An alternative option we considered was not modifying the decoder, but instead modifying our function unit circuit so that our Opcodes would be compatible with our function select codes. We ultimately decided against this alternative option, as even if we modified the function unit circuit, our function select codes would not be fully compatible with the Opcodes, and our final design would not have been able to meet all of the requirements described in the specification or perform all of the operations it was intended to perform. This alternative option also would have led to inefficiencies in the designs transistor count and propagation delay. If we didn't change the decoder our function unit would not work as a Simple Computer as outlined in this Learning Experience.

Section III: Simulation

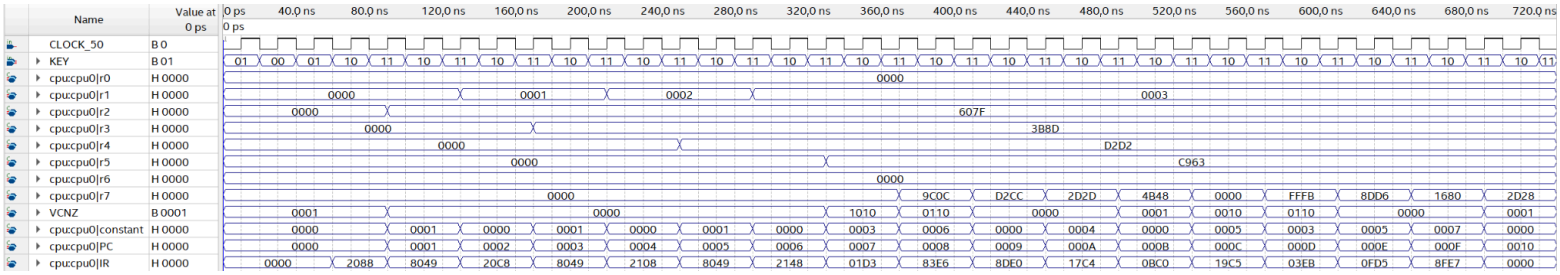

Figure 1.2 Simulation Of Our Design Implementation

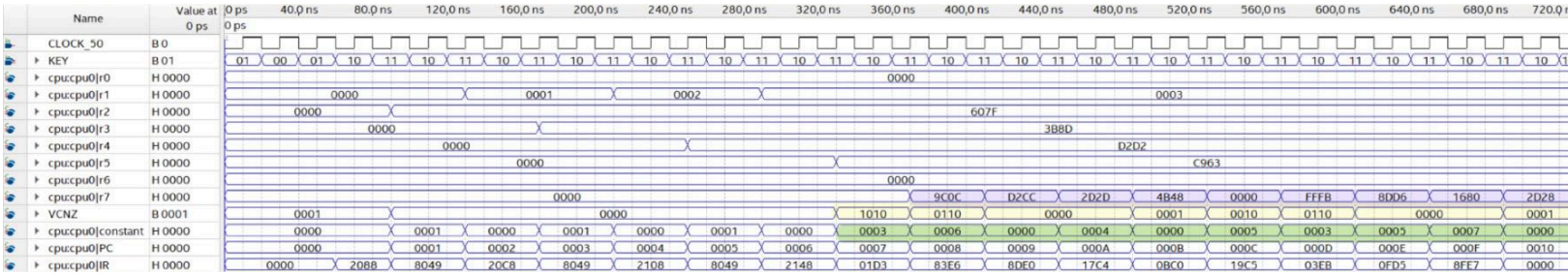

Figure 1.3 Annotated Simulation Of Our Design Implementation

Our circuit produced the expected outputs according to the Spec sheet and our own checks. We did notice the status bits were shifted over to the left from where they match up to the output of register 7. We annotated the simulation so that it would be easier to see where each of the values started for checking, and reading over our simulation results.

Section IV: Evaluation of Implementation

Assessment Of Operation Set Selection

We chose our operation set based on how our opcodes corresponded with the decoder. We modified the decoder so that the function select bit [0] would be equal to Opcode bit [9]. The provided decoder was originally designed so the function select bit [0] would be equal to ((Opcode[15]&Opcode[14])'&Opcode[9]). This affected how we chose Opcode bit [9[in our operation set. To ensure our Opcodes would be properly read by the decoder, we also had to ensure that Opcode MB was equal to Opcode[15], MD was equal to Opcode[13], JB was equal to Opcode[13], BC was equal to Opcode[9], RW was equal to Opcode[14]', MW was equal to (Opcode[14]&Opcode[15]') and PL was equal to (Opcode[14]&Opcode[15]. These were the factors that influenced how we selected our operation set and the modifications we had to make to our design so the implementation did not negatively affect the capability of the resulting Simple Computer. If we didn't change the decoder our function unit would not work as a Simple Computer as outlined in this Learning Experience. If we were to redesign we would choose more functions that involve strictly operand A because the mux B select in the opcode caused compatibility issues with the function select, and these compatibility issues were what we resolved by modifying the decoder. Our processor as designed is capable of performing all the operations necessary to make it useful.

Transistor Count

Function Unit Circuit									
Gates / Elements	2AND	3AND	2OR	4x1 Mux	2x1 Mux	NOT	3OR	4AND	16AND
Number of Gates	1	5	2	1	3	12	1	2	1
Transistor Count	6	40	12	62	60	24	8	20	34
Total Transistors	266								

Figure 1.4 Function Unit Transistor Count

Arithmetic Block Full Circuit			
Gates / Elements	XOR	4x1 Mux	Full Adder
Number of Gates	1	26	16
Transistor Count	12	1612	1152
Total Transistors	2776		

Figure 1.5 Arithmetic Block Transistor Count

Logic Block Full Circuit					
Gates/ Elements	8x1 mux	NOT	NOR	AND	NAND
Number of Gates	1	2	1	1	1
Transistor Count	104	4	4	6	4
Total Transistors	122				

Figure 1.6 Logic Block Transistor Count

Shift Block Full Circuit	
Gates/Elements	2x1 Mux
Number of Gates	1
Transistor Count	20
Total Transistors	20

Figure 1.7 Shift Block Transistor Count

Busy System Transistor Count				
Circuit Element	3AND	2OR	NOT	4x1 mux
Number Of Elements	4	4	2	2
Transistor Count	32	24	8	124
Total Transistors	188			

Figure 1.8 Bus System Transistor Count

Total Circuit Transistor Count					
Circuit Element	Function Unit	Arithmetic Block	Logic Block	Shift Block	Bus System
Number Of Elements	1	1	1	1	1
Transistor Count	266	2776	122	20	188
Total Transistors	3372				

Figure 1.9 Total Circuit Transistor Count

Our final circuit was the same from Learning Experience F.1, so our transistor count remained the same. Our design consisted of a circuit for the function unit, an arithmetic block, a logic block, a shift block, and a bus system. Our transistor counts for the function unit circuit (*Figure 1.4*), arithmetic block (*Figure 1.5*), logic block (*Figure 1.6*), the shift block (*Figure 1.7*), and the bus system (*Figure 1.8)* were all added together to find the total transistor count shown in *Figure 1.9*. It is important to note that we had previously calculated that a 2x1 mux has 20 transistors, a 4x1 mux has 62 transistors, and a 8x1 mux has 104 transistors. These 'assumptions' are shown in our calculations of final transistor counts. Our design in Learning Experience F.2 was efficient, as we did not increase our transistor count from Learning Experience F.1. Our function unit and bus system in Learning Experience F.2 meets all of the

criteria in the specification and has a reasonably efficient transistor count. We also considered making design changes to our function unit circuit, but ultimately decided against it because it would have increased the transistor count. We ended up deciding to modify the decoder instead to keep our efficiently designed function unit and bus system.

Propagation Delay

Total Circuit Propogation Delay						
Gates/ Elements	NOT	4x1 Mux	Full Adder To Carry	Full Adder to Sum	2OR	3AND
Number of Gates	2	3	15	1	1	1
Propogation Delay (ps)	50	450	2625	250	75	100
Total Propogation Delay (ps)	3550					

Figure 1.10 Total Circuit Propagation Delay

Since our function unit and bus system used the same design as learning Experience F.1, our propagation delay also remained the same. We determined that the critical path of our circuit would have been from one of our operand values via the bus system to an arithmetic block. We previously calculated the propagation delays of a 4x1 mux to be 150 ps, full adder to carry to be 175ps, and full adder to sum to be 250ps. These ‘assumptions’ are shown in our calculation of the total propagation delay in *Figure 1.10*. We also used the assumption that every two transistors accounted for a 25 ps propagation delay. Since our propagation delay did not increase from Learning Experience F.1, we deemed that our design process was efficient and justified. Our function unit and bus system in Learning Experience F.2 meets all of the criteria in the specification and has a reasonably efficient propagation delay.

Section V: Observations and Conclusions

Results

Our results are consistent with the requirements in the specification. We implemented a function unit in the Datapath of the Simple Computer and assigned Opcodes to instructions that use the function unit. Our function unit accepts two 16-bit 2's complement operands, a 4-bit function select code, and outputs the result of one of the thirteen operations with four status bits: V (overflow), C (carry), N (negative), Z (zero). We properly implemented our function unit design in the Simple Computer CPU and assigned opcodes to instructions that use the function unit. Our results are consistent with the requirements in the specification. The design for this function unit was implemented using only permitted Verilog practices, and followed all Verilog guidelines outlined in the specification. We also did an adequate job reducing FPGA resources, which we outlined in our discussion in Section III and all of our instructions are completed in one clock cycle. We followed the instruction format in Section 2.3 of the specification sheet. We also only modified the permitted files: function_unit.v, instr_decoder.v, data.txt, instruction.txt, and waveform files. We also ensured the original instructions, in addition to our new instructions and Opcodes.

Conclusions

The most significant learning experience we obtained from Learning Experience F.2 was learning how to be adaptable. When we originally approached our design for this project, we planned on keeping the same function unit from Learning Experience F.1 and just designing our Opcodes so they would be compatible with the decoder provided to us in the datapath of this learning experience. We quickly realized that our function unit design did not allow for our Opcodes to be compatible with the provided decoder. We tried to adapt our design approach and modify our function unit so that we would be able to design Opcodes that would be compatible with the decoder, but we kept encountering trouble with compatibility as we attempted this, and this approach also increased our FPGA resource usage which was one of our criteria for success and a constraint in the specification of this learning experience. Between our struggles with compatibility and inability to create an efficient design when modifying our function unit, we decided that we, yet again, needed to try a different approach. We then adapted our design approach again and decided to modify the decoder and keep the function unit the same from Learning Experience F.1. This approach finally worked and allowed us to meet our criteria for success and all of the requirements and constraints outlined in the specification for Learning Experience F.2. Through this process, we learned that if we remain adaptable in our design approach in the field of Electrical and Computer Engineering, our chances of success dramatically increase. The experience designing the function unit in the Datapath of the Simple Computer with instructions also provided us with valuable experience in seeing how smaller designs can be incorporated into a larger system. Having smaller parts working together efficiently in a system is a critical concept in the field of Electrical and Computer Engineering. The fields of electrical engineering, computer engineering, computer

science, software engineering, and so on have lots of overlap, and many systems within these fields also incorporate multiple, smaller aspects of these other fields working together in a larger system. Overall, between the experience with adaptability in project design and the experience with incorporating smaller designs into a larger system, we had many valuable learning experiences in Learning Experience F.2